

Лекция 9. Основы Docker и контейнеризация приложений

Docker помогает запускать приложения в изолированных контейнерах с повторяемой конфигурацией

Контейнеризация не отменяет администрирование Linux, сети, хранения данных и безопасности

Учебный сценарий: развернуть Nginx, подключить данные, собрать образ и описать запуск через Compose

Главная идея: контейнер должен быть воспроизводимым, диагностируемым и безопасно ограниченным

Цель лекции

- Понять отличие контейнера от виртуальной машины
- Уметь объяснить Docker Engine, Docker Daemon, Docker API, Registry, Image и Container
- Уметь безопасно запускать контейнер, публиковать порт и подключать данные
- Уметь читать logs, выполнять exes, задавать restart policy и healthcheck
- Уметь составить Dockerfile и compose.yaml для базового web-сервиса

Учебные вопросы

1. Контейнер и виртуальная машина; Docker Engine, Client и Registry
2. Image, container, слои и жизненный цикл
3. Запуск, остановка, restart и удаление; публикация портов и env
4. Volumes, bind mounts и сети
5. Logs и exec; restart policy
6. Dockerfile и Docker Compose
7. Root, privileged mode и доверие к образам

Учебный сценарий лекции

Получить образ `nginx:stable-alpine`

Запустить Nginx только на `localhost`-адресе хоста:
`127.0.0.1:8080`

Подключить HTML-каталог как `read-only bind mount`

Собрать собственный образ `lab-nginx:1.0`

Перенести запуск в `compose.yaml` и проверить отказ,
`restart`, `logs` и `inspect`

1. Контейнер и виртуальная машина

Контейнер — это изолированный процесс с файловым окружением, сетью и ограниченными ресурсами

Контейнер использует ядро хоста и не содержит отдельное ядро гостевой ОС

Виртуальная машина имеет собственную гостевую ОС и виртуальное оборудование

Контейнер обычно запускается быстрее и потребляет меньше ресурсов

Граница изоляции контейнера не равна полноценной границе виртуальной машины

Linux-контейнеру нужен Linux Kernel: поверх Windows

Изоляция контейнера строится на механизмах ядра Linux

Namespaces разделяют процессы, сеть, mount points и другие пространства имен

Cgroups ограничивают и учитывают CPU, RAM и другие ресурсы

Файловая система контейнера строится из слоев image и writable layer

Capabilities ограничивают часть прав root-процесса

Контейнер остается процессом на хосте, а не отдельным сервером внутри сервера

Схема: виртуальная машина и контейнер

Два подхода к изоляции приложений

ВИРТУАЛЬНАЯ МАШИНА



КОНТЕЙНЕР



Быстрее запуск

Контейнеры запускаются за секунды, так как не загружают гостевую ОС



Меньше ресурсов

Контейнеры используют общее ядро хоста и требуют меньше RAM и CPU



Больше плотность

На одном хосте можно запустить больше контейнеров, чем виртуальных машин



Изоляция

VM: сильная изоляция на уровне гипервизора
Контейнер: изоляция на уровне ядра Linux



Запуск

VM: десятки секунд и больше
Контейнер: доли секунд



Ресурсы

VM: больше накладных расходов
Контейнер: меньше накладных расходов



Управление

VM: сложнее, больше компонентов
Контейнер: проще, меньше компонентов



Образ

VM: образ включает гостевую ОС
Контейнер: образ только с приложением



Важно

Контейнер ≠ VM
Контейнеры не заменяют виртуальные машины, а дополняют их



Итог:

Контейнер использует ядро хоста и обеспечивает легковесную изоляцию процессов, файловой системы, сети и ресурсов. Виртуальная машина содержит полноценную гостевую ОС и обеспечивает более сильную изоляцию на уровне гипервизора.

Docker состоит из Client, API, Daemon и Runtime

docker CLI принимает команды администратора

Docker API передает команды Docker Daemon

dockerd управляет images, containers, networks и volumes

Container runtime запускает контейнерные процессы

Registry хранит и отдает images, например Docker Hub или внутренний registry

Docker Socket дает очень широкие права

Docker Socket обычно расположен здесь:

`/var/run/docker.sock`

Пользователь с доступом к socket может управлять

Docker Daemon

Группа docker не является обычной безопасной группой для новичка

Команда `sudo usermod -aG docker student` должна рассматриваться как выдача повышенных полномочий

Доступ к socket нужно ограничивать так же

внимательно, как административный доступ к хосту

2. Image, container, слои и жизненный цикл

Image — шаблон файловой системы и метаданных для запуска контейнера

Container — экземпляр image с собственным состоянием запуска

Writable layer — изменяемый слой конкретного контейнера

Image ID или digest фиксируют конкретное содержимое

Tag — удобная ссылка, которая может быть переназначена на другой image

Тег удобен, но не гарантирует неизменяемость

nginx:latest сегодня и через месяц может указывать на разные images

Для лаборатории лучше использовать фиксированный tag: nginx:stable-alpine или nginx:1.27-alpine

Для максимальной воспроизводимости используют digest: nginx@sha256:...

Проверка images: docker image ls

Подробности image: docker image inspect nginx:stable-alpine

Writable layer сохраняется после stop и start

docker stop web останавливает контейнер, но не удаляет writable layer

docker start web запускает тот же контейнер с прежним writable layer

docker restart web перезапускает существующий контейнер

docker rm web удаляет контейнер и его writable layer

Для важных данных writable layer использовать не следует

docker run создает новый контейнер

Команда `docker run` не запускает ранее
остановленный контейнер

`docker run` создает новый container из image и
запускает его

`docker start` запускает уже существующий container

`docker restart` перезапускает уже существующий
container

Типичная ошибка: после остановки выполнить `docker run` и получить второй контейнер или конфликт порта

Просмотр жизненного цикла контейнеров

`docker ps` показывает только запущенные containers

`docker ps -a` показывает запущенные и остановленные containers

`docker inspect web` показывает подробную конфигурацию контейнера

`docker rm -f web` принудительно останавливает и удаляет container

`docker container prune` удаляет остановленные containers и требует осторожности

Контейнер живет, пока работает PID 1

Контейнер останавливается, когда завершается главный процесс внутри него

Команда `docker run ubuntu echo hello` выполнит `echo` и сразу завершит контейнер

`-d` запускает контейнер в фоне, но не делает процесс бесконечным

Главный процесс должен корректно обрабатывать сигналы

Это важно для штатного завершения баз данных и очередей

docker stop сначала отправляет SIGTERM

docker stop отправляет главному процессу SIGTERM

Затем Docker ожидает заданное время

Если процесс не завершился, отправляется SIGKILL

Приложение должно успеть закрыть соединения,
транзакции и дочерние процессы

Для чувствительных сервисов нужно проверять
поведение остановки заранее

3. Запуск, остановка, restart, порты и env

`docker pull` получает image из registry

`docker run` создает и запускает новый container

`docker stop`, `start` и `restart` управляют существующим container

Публикация портов связывает порт хоста с портом контейнера

Environment variables передают настройки приложению, но плохо подходят для секретов

Безопасный первый запуск Nginx

Команда: `docker pull nginx:stable-alpine`

Назначение: получить фиксированный образ Nginx для лаборатории

Команда: `docker run --name web -d -p 127.0.0.1:8080:80 nginx:stable-alpine`

Назначение: опубликовать Nginx только на loopback-адресе хоста

Проверка: `curl http://127.0.0.1:8080/`

Публикация порта может открыть сервис наружу

Команда `-p 8080:80` часто публикует порт на всех интерфейсах хоста: `0.0.0.0:8080`

Так сервис может стать доступен не только локально

Для локальной лаборатории безопаснее: `-p 127.0.0.1:8080:80`

Проверка Docker: `docker port web`

Проверка Linux: `ss -ltnp | grep 8080`

EXPOSE и -p решают разные задачи

EXPOSE 80 в Dockerfile документирует предполагаемый порт приложения

EXPOSE сам по себе не публикует порт на хосте

-p 8080:80 создает реальную публикацию порта

Контейнеры в одной Docker network могут общаться без публикации порта наружу

Database port обычно не публикуют, если к базе

обращается только приложение внутри сети проекта

Environment variables не подходят для секретов

Несекретный пример: `docker run -e APP_ENV=lab nginx:stable-alpine`

Environment variables могут попасть в shell history

Их можно увидеть через `docker inspect web`

Они могут попасть в `compose config`, CI logs или дампы процесса

Пароли лучше передавать через `secret file`, `secret manager` или защищенный конфигурационный файл

4. Volumes, bind mounts и networks

Volume хранит данные вне writable layer контейнера

Bind mount подключает конкретный путь хоста внутрь контейнера

Anonymous volume создается без понятного имени и его легко потерять из вида

User-defined network дает контейнерам встроенное DNS-разрешение по имени

Данные контейнерного приложения требуют отдельной стратегии backup

Named volume переживает удаление контейнера

Команда: `docker volume create webdata`

Назначение: создать именованное хранилище данных

`docker rm web` обычно не удаляет `named volume`

`docker volume rm webdata` удалит `named volume` и его данные

Проверка: `docker volume inspect webdata`

docker compose down -v может уничтожить данные

docker compose down удаляет containers проекта и обычную сеть проекта

Named volumes обычно сохраняются

Bind-mounted data на хосте также сохраняются

docker compose down -v дополнительно удаляет volumes проекта

Перед down -v нужно понимать, какие данные будут потеряны

Volume — это не backup

Volume сохраняет данные между пересозданиями контейнера

Но volume не защищает от удаления, повреждения или шифрования данных

Для баз данных нужна консистентная процедура: dump, snapshot или штатный backup tool

Нужно учитывать владельцев, права, шифрование, retention и test restore

Backup volume проектируется отдельно от docker run или compose.yaml

Bind mount может дать контейнеру опасный доступ

Безопасный пример: `--mount type=bind,source=/srv/www,target=/usr/share/nginx/html,readonly`

Опасный пример: `-v /:/host`

Особо опасный пример: `-v /var/run/docker.sock:/var/run/docker.sock`

Bind mount связывает контейнер с реальной файловой системой хоста

Чем шире source path, тем выше риск для хоста

UID и GID влияют на доступ к bind mount

Процесс внутри контейнера использует числовые UID/GID ядра

Контейнерный пользователь UID 1000 может не иметь прав на каталог хоста

Контейнерный root при обычной модели может писать как UID 0 на bind mount

Проверка внутри контейнера: `docker exec web id`

Проверка на хосте: `ls -ln /srv/www`

User-defined network дает DNS по имени контейнера

Команда: `docker network create appnet`

Назначение: создать пользовательскую bridge-сеть

Команда: `docker run -d --name web --network appnet
nginx:stable-alpine`

Другой контейнер в `appnet` может обращаться к
`http://web/`

Docker Compose автоматически создает сеть проекта,
если не задана другая

Database port обычно не публикуют наружу

Внутри Docker network приложение может обращаться к базе по имени service

Пример: web обращается к db:5432

Публикация 5432:5432 увеличивает поверхность атаки

Если база нужна только приложению, секция ports для db не требуется

Для диагностики лучше использовать отдельный временный доступ или внутренний контейнер-клиент

Схема: ports, volumes и user-defined network

Как контейнеры получают доступ извне, сохраняют данные и общаются между собой



PORTS – доступ к контейнеру извне

Публикация порта хоста к порту контейнера



Пример команды

```
docker run -d \
  --name web \
  -p 127.0.0.1:8080:80 \
  nginx:stable-alpine
```

`-p <host_ip:host_port>:<container_port>`

- 127.0.0.1:8080 – доступ только с хоста
- Внешний порт 8080 проброшен во внутрь контейнера на 80/tcp
- Без `-p` сервис внутри контейнера недоступен извне

Проверка

```
> curl http://127.0.0.1:8080
```

Откройте в браузере <http://127.0.0.1:8080>



Полезно знать

- По умолчанию контейнеры изолированы и недоступны извне
- Публикуйте только нужные порты
- Привязывайте к 127.0.0.1 для локального доступа
- Используйте firewall/ufw на хосте для дополнительной защиты



VOLUMES – сохранение и обмен данными

Данные живут вне контейнера и не теряются при удалении



Варианты монтирования



Named Volume (рекомендуется для данных приложений)

```
docker run -d --name web \
  -p 127.0.0.1:8080:80 \
  -v webdata:/usr/share/nginx/html:ro \
  nginx:stable-alpine
```

Docker управляет расположением тома на хосте (обычно /var/lib/docker/volumes/...)



Bind Mount (для разработки и конфигов)

```
docker run -d --name web \
  -p 127.0.0.1:8080:80 \
  -v /opt/lab/html:/usr/share/nginx/html:ro \
  nginx:stable-alpine
```

Использует папку на хосте напрямую (изменения видны с обеих сторон)

Проверка

```
> docker volume ls
ls -la /opt/lab/html
```

Важно

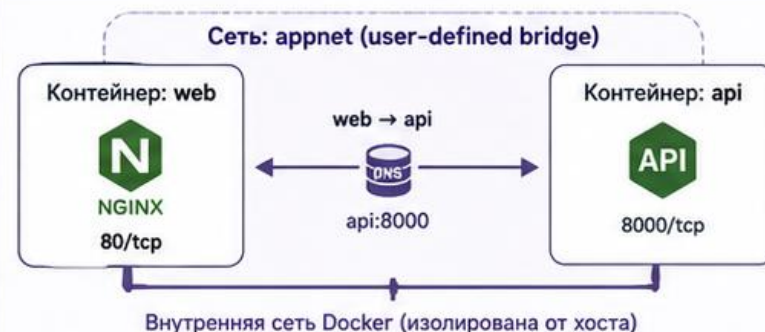


- Монтируйте только нужные пути
- Используйте `:ro` (read-only) где возможно
- Бэкапы делайте с томов, а не из контейнера



USER-DEFINED NETWORK – сеть для контейнеров

Контейнеры общаются по именам, а не по IP-адресам



Пример команд

- Создаем сеть
`docker network create appnet`
- Запускаем контейнеры в этой сети
`docker run -d --name api --network appnet api:1.0`
`docker run -d --name web --network appnet -p 127.0.0.1:8080:80 nginx`
- Проверяем связь по имени
`docker exec web wget -qO- http://api:8000/health`

Преимущества user-defined сети



- Встроенный DNS: контейнеры обращаются по именам
- Изоляция: трафик не виден другим сетям по умолчанию
- Удобное управление и подключение к нужным сетям
- Поддержка драйверов (bridge, overlay и др.)

Итог: три ключевых механизма для практических сценариев



Ports – даем доступ извне

Публикуем только нужные порты и ограничиваем доступ (например, 127.0.0.1 или через firewall).



Volumes – сохраняем данные

Данные живут вне контейнера и не теряются. Используем named volumes или bind mounts.



User-defined network – соединяем сервисы

Контейнеры общаются по именам в изолированной сети. Просто, надежно и безопасно.

Команды на каждый день

```
> docker ps
docker logs -f <container>
docker inspect <container/network/volume>
docker rm -f <container>
```

5. Logs, exec и restart policy

`docker logs` показывает `stdout` и `stderr` главного процесса

`docker exec` запускает дополнительную команду внутри работающего контейнера

`Restart policy` определяет автоматический запуск после завершения контейнера

`Healthcheck` проверяет логическое состояние приложения

Ручные изменения через `exec` не должны становиться способом настройки `production`

Container logs нужно ограничивать ротацией

Команда: `docker logs --tail 100 web`

Назначение: показать последние строки журнала

Команда: `docker logs --since 10m web`

Назначение: показать события за последние 10 минут

Без ротации JSON logs могут заполнить диск хоста

Logging driver влияет на docker logs

Частые drivers: json-file, local, journald, syslog

docker logs читает stdout/stderr, если выбранный driver поддерживает такой просмотр

Для Compose полезно задать json-file rotation

Параметры: max-size: '10m' и max-file: '3'

Логи контейнера не заменяют централизованное журналирование

docker exec нужен для диагностики, а не ручной настройки

Команда: `docker exec -it web sh`

Назначение: открыть shell внутри работающего контейнера, если shell есть в image

Минимальный image может не содержать bash, sh, curl, ps или package manager

Ручное редактирование через exec исчезнет после пересоздания контейнера

Постоянная настройка должна быть в Dockerfile, config file или compose.yaml

Restart policy имеет разные режимы

no — не перезапускать автоматически

on-failure — перезапускать при ненулевом exit code

always — перезапускать после остановки и запуска

Docker Daemon

unless-stopped — перезапускать, кроме контейнера, остановленного администратором

Restart policy реагирует на завершение главного процесса, а не на HTTP 500 внутри приложения

Healthcheck проверяет состояние приложения

Dockerfile пример: HEALTHCHECK --interval=30s --timeout=3s CMD wget -q --spider http://localhost/ || exit 1

Compose может задавать healthcheck в сервисе

Проверка: docker inspect --format

'{{.State.Health.Status}}' web

Статус unhealthy не всегда означает автоматический restart самим Docker Engine

Healthcheck полезен для мониторинга и оркестраторов

6. Dockerfile и Docker Compose

Dockerfile описывает сборку собственного image

Compose описывает запуск одного или нескольких сервисов

Dockerfile и compose.yaml должны храниться в Git

Image отвечает за содержимое и команду запуска

Compose отвечает за параметры deployment: ports, volumes, networks, restart, logging и limits

Dockerfile должен описывать и файлы, и метаданные

FROM задает базовый image

WORKDIR задает рабочий каталог

COPY добавляет файлы в image

EXPOSE документирует порт приложения

ENTRYPOINT и CMD задают команду запуска

USER помогает запускать приложение без root, если image к этому готов

Пример Dockerfile для учебного Nginx

```
FROM nginx:stable-alpine  
WORKDIR /usr/share/nginx/html  
COPY index.html ./index.html  
EXPOSE 80  
HEALTHCHECK --interval=30s --timeout=3s CMD wget -q --  
spider http://localhost/ || exit 1
```

Для официального nginx base image команда запуска уже определена

.dockerignore защищает build context от лишнего

Build context может случайно включить .git, logs, backups, PCAP и secrets

Пример строк: .git, *.pcap, *.log, *.bak, .env, secrets/

.dockerignore уменьшает размер build context

.dockerignore не является полноценным secret management

Секреты не должны попадать в build context вообще

Порядок инструкций влияет на build cache

RUN, COPY и ADD создают файловые изменения

Некоторые инструкции задают metadata

BuildKit может оптимизировать процесс сборки

Если сначала COPY . /app, любое изменение файла может сбросить cache следующих шагов

Сначала копируют файлы зависимостей,
устанавливают зависимости, затем копируют
остальной код

Сборка image и проверка tag

Команда: `docker build -t lab-nginx:1.0 .`

Назначение: собрать image с локальным tag

Повторная сборка с тем же tag переназначит tag на новый image ID

Проверка: `docker image ls`

Подробности: `docker image inspect lab-nginx:1.0`

compose.yaml: сервис web

services:

web:

image: nginx:stable-alpine

container_name: web

ports:

- '127.0.0.1:8080:80'

compose.yaml: данные, restart и filesystem

volumes:

- './html:/usr/share/nginx/html:ro'

restart: unless-stopped

read_only: true

tmpfs:

- /var/cache/nginx
- /var/run

compose.yaml: logs, healthcheck и security

logging: driver json-file, max-size '10m', max-file '3'

healthcheck: wget -q --spider http://localhost/

security_opt:

- no-new-privileges:true

Если read_only мешает конкретному image, нужно явно добавить tmpfs или изменить конфигурацию

Compose-файл проверяют командой docker compose config

Команды Compose для жизненного цикла проекта

`docker compose up -d` создает и запускает сервисы проекта

`docker compose ps` показывает состояние сервисов

`docker compose logs web` показывает журналы web-сервиса

`docker compose down` удаляет containers и обычную сеть проекта

`docker compose down -v` дополнительно удаляет volumes проекта и может уничтожить данные

depends_on не означает готовность приложения

depends_on задает порядок запуска сервисов

Он не всегда означает, что база данных уже принимает соединения

Для готовности применяют healthcheck

Приложение должно иметь retry logic при подключении к зависимости

В production готовность сервиса проверяется

прикладным тестом, а не только порядком запуска

7. Root, privileged mode и доверие к образам

Root внутри контейнера ограничен namespaces, capabilities и настройками runtime

Но root в rootful Docker остается привилегированным процессом с точки зрения ядра

Опасные mounts или уязвимость runtime могут затронуть хост

Privileged mode снимает важные ограничения

Доверие к image, registry и compose-файлу является частью безопасности

Non-root запуск требует подготовки image

Dockerfile пример: RUN addgroup -S app && adduser -S app -G app

Dockerfile пример: USER app

Compose вариант: user: '1000:1000'

Нужно проверить права файлов и каталогов записи

Нельзя просто указать USER и ожидать, что любое приложение заработает

Capabilities и no-new-privileges безопаснее privileged mode

--privileged выдает контейнеру слишком широкие
ВОЗМОЖНОСТИ

Более точный подход: --cap-drop ALL и выборочный --
cap-add

Можно подключить только нужное устройство через --
device

Полезны read-only filesystem, seccomp, AppArmor или
SELinux

Compose-настройка no-new-privileges снижает риск
повышения прав внутри контейнера

Mount Docker Socket почти дает контроль над хостом

Опасный mount: -v

`/var/run/docker.sock:/var/run/docker.sock`

Контейнер сможет управлять Docker Daemon

Он может запускать новые containers и подключать каталоги хоста

Он может создавать privileged containers

Такой mount допускается только при ясной архитектурной необходимости и дополнительных ограничениях

Официальный image не означает автоматическую безопасность

Даже официальный image может содержать известные уязвимости

Tag может изменить содержимое со временем

Нужны фиксированная версия, vulnerability scan и регулярная пересборка

Полезны digest, image signing и контролируемый registry

Хост Docker и container runtime также нужно обновлять

Контейнеру нужно задавать лимиты ресурсов

Без лимитов один контейнер может занять значительную часть ресурсов хоста

Пример: `docker run --memory 256m --cpus 0.5`

`nginx:stable-alpine`

Compose: `mem_limit: 256m`

Compose: `cpus: 0.5`

Поддержка отдельных параметров зависит от runtime и режима Compose

Мониторинг контейнеров включает host и приложение

Проверять Docker Daemon и состояние Running/Exited

Проверять restart count, health status, CPU и RAM

Проверять размер logs и docker system df

Проверять доступность приложения через HTTP или другой реальный протокол

Проверять состояние backup для volumes и прикладных данных

Команды prune могут удалить нужные объекты

`docker system prune` удаляет неиспользуемые объекты и build cache

`docker volume prune` удаляет неиспользуемые volumes

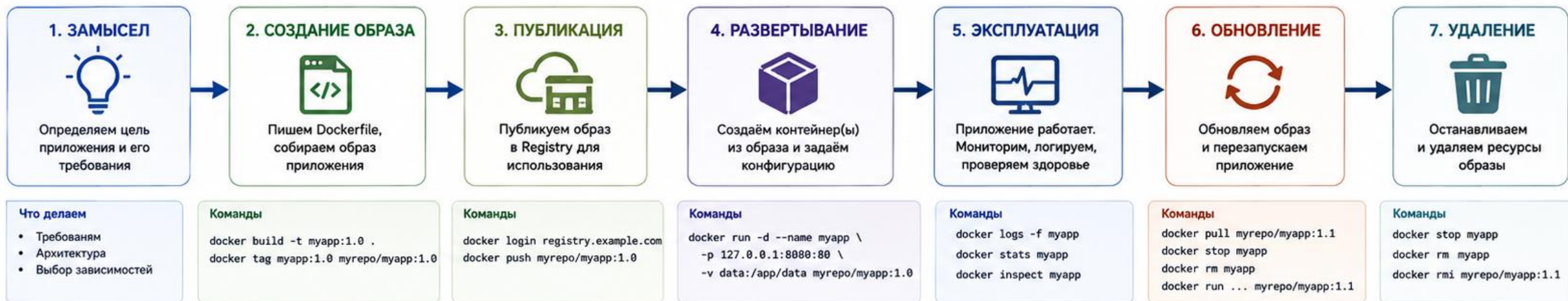
Перед prune нужно посмотреть `docker ps -a`, `docker image ls` и `docker volume ls`

Нельзя запускать prune как универсальное лечение нехватки места

Сначала понять, какие данные можно безопасно удалить

Диаграмма: жизненный цикл Docker-приложения

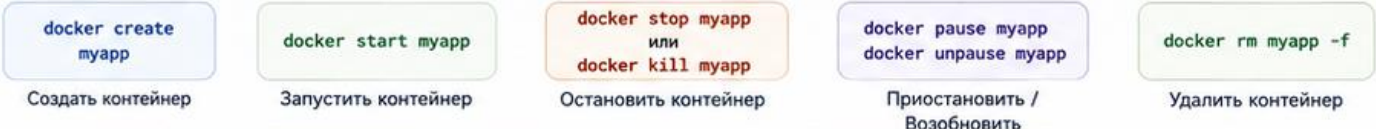
Путь от идеи до рабочего сервиса и обратно



СОСТОЯНИЯ КОНТЕЙНЕРА



Основные операции



ПОДКЛЮЧАЕМЫЕ РЕСУРСЫ



АРТЕФАКТЫ ДОКЕР-ПРИЛОЖЕНИЯ



Проверка и диагностика

- `docker ps -a` – список контейнеров
- `docker logs` – просмотр логов
- `docker exec -it myapp sh` – команды внутри
- `docker inspect myapp` – детальная информация
- `docker top myapp` – процессы в контейнере



Полезные практики

- Используйте образы с фиксированными тегами или digest
- Минимизируйте образ (alpine, multi-stage build)
- Не запускайте от root, задавайте пользователя
- Ограничивайте ресурсы (`--cpus`, `--memory`)
- Настройте healthcheck и restart policy



Пример healthcheck в Dockerfile

```
HEALTHCHECK --interval=30s --timeout=5s \
--retries=3 CMD curl -f http://localhost/health \
|| exit 1
```

Обозначения

- Переход к следующему шагу
- - - - - Возможный возврат к предыдущему шагу



Главная идея

Контейнеры – это неизменяемые артефакты. Обновляйте через создание новых образов и замену контейнеров.

Лабораторный цикл Build, Run, Inspect, Diagnose, Recreate

Run: запустить nginx:stable-alpine на 127.0.0.1:8080

Inspect: проверить docker ps, docker port, docker inspect и ss -lntp

Diagnose: проверить docker logs, docker exec и curl

Recreate: удалить container и убедиться, что bind mount или volume сохраняет данные

Build and Compose: собрать image и перенести запуск в compose.yaml

Приемочная матрица лабораторной работы

`docker version` показывает Client и Server

`curl 127.0.0.1:8080` возвращает страницу Nginx

`docker port web` показывает публикацию только на 127.0.0.1

`docker logs web` показывает HTTP-запросы

`docker inspect` подтверждает отсутствие privileged mode

`docker compose down` сохраняет данные, а `risk down` - v объяснен студентом

Итоги лекции

Контейнер — изолированный процесс, использующий ядро хоста

`docker run` создает новый container, а `docker start` запускает существующий

Публикацию портов, `env`, `volumes`, `bind mounts` и `networks` нужно проектировать осознанно

`Dockerfile` и `compose.yml` делают запуск повторяемым, если в них включены данные, `logs`, `healthcheck` и ограничения

Безопасность Docker зависит от `socket`, `root`, `capabilities`, `mounts`, `image source` и лимитов ресурсов

Контрольные вопросы

Почему контейнер не является полноценной виртуальной машиной?

Чем tag отличается от image digest?

Почему -p 8080:80 может быть опаснее, чем -p 127.0.0.1:8080:80?

Чем named volume отличается от bind mount и anonymous volume?

Почему docker compose down -v опасен для данных?

Какие риски создает mount /var/run/docker.sock внутрь контейнера?